DJANGO MAJOR APPLICATION • MINI PROJECT

ENTERPRISE TIER

QuizCraft AI Examination System

Interactive Assessment Platform, Student Dashboard & Comprehensive Admin Console

PROJECT NAME	APP NAME	USER ROLES	DATABASE SCHEMA
mini_project	**ai_quiz**	**Admin & Student Portals**	**Quizzes, Questions, Results**

Executive Summary: The QuizCraft AI Assessment platform delivers an interactive examination environment built on Django. It supports dual role experiences: Students can explore topics (Python, Java, C++, etc.), attempt timed multi-choice questions with automated grading, and review historical performance. Administrators command an executive control room to create quizzes, curate question pools, modify options, and inspect all student submissions in real time.

1. Relational Data Architecture (`ai_quiz/models.py`)

The system establishes a three-tier relational schema interconnecting quizzes, question items, and student exam attempts.

Model	Field	Type / Relationship	Description
Quiz	`name`	CharField(max_length=100)	Quiz subject or examination title
	`number_of_questions`	IntegerField	Target question threshold per exam session
Question	`quiz`	ForeignKey(Quiz, CASCADE)	Parent exam topic relationship
	`question_text`	CharField(max_length=500)	Problem statement presented to candidate
	`option1 – 4`	CharField(max_length=200)	Four multiple choice options
	`correct_answer`	CharField(max_length=200)	Verified key utilized for automatic score grading
TestResult	`user`	ForeignKey(User, CASCADE)	Student candidate who attempted the exam
	`quiz`	ForeignKey(Quiz, CASCADE)	Associated subject category
	`score`	IntegerField	Total verified correct answers
	`total_questions`	IntegerField	Denominator question count
	`attempted_at`	DateTimeField(auto_now_add=True)	Timestamp for audit trail and history logs

```python
from django.db import models
from django.contrib.auth.models import User


class Quiz(models.Model):
    name = models.CharField(max_length=100)
    number_of_questions = models.IntegerField()

    def __str__(self):
        return self.name


class Question(models.Model):
    quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE)

    question_text = models.CharField(max_length=500)
```

```python

    option1 = models.CharField(max_length=200)
    option2 = models.CharField(max_length=200)
    option3 = models.CharField(max_length=200)
    option4 = models.CharField(max_length=200)

    correct_answer = models.CharField(max_length=200)

    def __str__(self):
        return self.question_text


class TestResult(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    quiz = models.ForeignKey(Quiz, on_delete=models.CASCADE)

    score = models.IntegerField()
    total_questions = models.IntegerField()

    attempted_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user.username} - {self.quiz.name}"
```

2. Application Routing & Endpoints

Routes are cleanly segregated into student examination flows, auth operations, and administrator management handlers.

Endpoint	View Method	Name	Access & Role
`''`	`views.landing`	`'landing'`	Public Landing Page
`'home/'`	`views.home`	`'home'`	Student Dashboard (Login Required)
`'signup/'`	`views.signup`	`'signup'`	Candidate Account Creation
`'login/'`	`auth_views.LoginView`	`'login'`	Django Authentication Form
`'quiz/<quiz_id>/'`	`views.start_quiz`	`'start_quiz'`	Interactive Exam Runner & Scoring
`'history/'`	`views.history`	`'history'`	Student Past Score History
`'admin-panel/'`	`views.admin_dashboard`	`'admin_dashboard'`	Admin KPI Metrics & Control Panel
`'admin-panel/add-quiz/'`	`views.add_quiz`	`'add_quiz'`	Create New Exam Subject
`'admin-panel/questions/'`	`views.manage_questions`	`'manage_questions'`	Repository of all questions
`'admin-panel/results/'`	`views.view_results`	`'view_results'`	All Students Submissions Audit Log

App Routing (`ai_quiz/urls.py`):

```python
from django.urls import path
from django.contrib.auth import views as auth_views

from . import views

urlpatterns = [
    path('', views.landing, name='landing'),

    path('home/', views.home, name='home'),

    path('signup/', views.signup, name='signup'),

    path('login/', auth_views.LoginView.as_view(template_name='login.html'), name='login'),

    path('logout/', views.logout_view, name='logout'),

    path('quiz/<int:quiz_id>/', views.start_quiz, name='start_quiz'),

    path('history/', views.history, name='history'),

    path('admin-panel/', views.admin_dashboard, name='admin_dashboard'),

    path('admin-panel/add-quiz/', views.add_quiz, name='add_quiz'),

    path('admin-panel/quiz/delete/<int:quiz_id>/', views.delete_quiz, name='delete_quiz'),

    path('admin-panel/add-question/', views.add_question, name='add_question'),

    path('admin-panel/questions/', views.manage_questions, name='manage_questions'),

    path('admin-panel/questions/edit/<int:question_id>/', views.edit_question, name='edit_question'),

    path('admin-panel/questions/delete/<int:question_id>/', views.delete_question, name='delete_question'),

    path('admin-panel/results/', views.view_results, name='view_results'),
]
```

Project Routing (`mini_project/urls.py`):

```python
"""
URL configuration for mini_project project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.1/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('ai_quiz.urls')),

]
```

3. Business Logic & Controller Functions (`ai_quiz/views.py`)

The controller coordinates student authentication, automated answer evaluation, score calculation, and administrative CRUD operations.

> **Key Functional Components:**
>
> - `start_quiz(request, quiz_id)`: Pulls randomized or sequential questions, evaluates POSTed candidate selections against `correct_answer`, and logs a persistent `TestResult` record.
> - `history(request)`: Queries past test attempts for the current user and computes highest personal score and total sessions.
> - `admin_dashboard(request)`: Aggregates real-time count metrics (Total Quizzes, Questions, Results) for management overview.
> - CRUD handlers: `add_quiz`, `delete_quiz`, `add_question`, `edit_question`, `delete_question`, and `view_results`.

```python
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login, logout as auth_logout
from django.contrib.auth.decorators import login_required
from .forms import SignupForm
from .models import Quiz, TestResult, Question

# Landing Page: This is the first page of our website
def landing(request):
    if request.user.is_authenticated:
        return redirect('home')
    return render(request, 'landing.html')


# Home Page: Shows a list of quizzes. User must be logged in.
@login_required
def home(request):
    quizzes = Quiz.objects.all().order_by('id')
    return render(request, 'home.html', {'quizzes': quizzes})


# Logout View: Handles both GET and POST requests safely without 405 errors
def logout_view(request):
    auth_logout(request)
    return redirect('landing')


# Signup Page: Allows new users to create an account
def signup(request):
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
        form = SignupForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user) # Automatically log the user in after signing up
            return redirect('home')
    else:
        form = SignupForm()
    return render(request, 'signup.html', {'form': form})


# Start Quiz: Where the user answers questions and gets a score
@login_required
def start_quiz(request, quiz_id):
    quiz = get_object_or_404(Quiz, id=quiz_id)
    questions = quiz.question_set.all().order_by('id')[:quiz.number_of_questions]
    total_q = questions.count()

    if request.method == 'POST':
        score = 0
```

```
        # Check every question to see if the chosen answer is correct
        for question in questions:
            selected_answer = request.POST.get(f'question_{question.id}')
            if selected_answer and selected_answer.strip() == question.correct_answer.strip():
                score += 1

        # Save the final score into our database
        TestResult.objects.create(
            user=request.user,
            quiz=quiz,
            score=score,
            total_questions=total_q
        )
        percentage = round((score / total_q) * 100) if total_q > 0 else 0
        return render(request, 'result.html', {
            'quiz': quiz,
            'score': score,
            'total': total_q,
            'percentage': percentage
        })

    # Show the quiz page with all questions
    return render(request, 'quiz.html', {
        'quiz': quiz,
        'questions': questions,
        'total_questions': total_q
    })


# History: Shows past test results for the current user
@login_required
def history(request):
    # Fetch results for the logged-in user, ordered by newest first
    results = TestResult.objects.filter(user=request.user).order_by('-attempted_at')
    total_attempts = results.count()
    best_score = max([r.score for r in results], default=0) if total_attempts > 0 else 0
    return render(request, 'history.html', {
        'results': results,
        'total_attempts': total_attempts,
        'best_score': best_score
    })


# Admin Dashboard: Main control panel for the site admin
def admin_dashboard(request):
    quizzes = Quiz.objects.all().order_by('-id')
    total_quizzes = quizzes.count()
    total_questions = Question.objects.count()
    total_results = TestResult.objects.count()
    return render(request, 'admin_dashboard.html', {
        'quizzes': quizzes,
        'total_quizzes': total_quizzes,
        'total_questions': total_questions,
        'total_results': total_results
    })


# Add Quiz: Create a new quiz topic
def add_quiz(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        number_of_questions = request.POST.get('number_of_questions')
        if name and number_of_questions:
            Quiz.objects.create(name=name.strip(), number_of_questions=int(number_of_questions))
            return redirect('admin_dashboard')

    return render(request, 'add_quiz.html')


# Delete Quiz: Remove a quiz
def delete_quiz(request, quiz_id):
    quiz = get_object_or_404(Quiz, id=quiz_id)
    quiz.delete()
    return redirect('admin_dashboard')

```

```python

# Add Question: Create a new question and assign it to a quiz
def add_question(request):
    quizzes = Quiz.objects.all().order_by('name')

    if request.method == 'POST':
        quiz_id = request.POST.get('quiz')
        if quiz_id:
            quiz = get_object_or_404(Quiz, id=quiz_id)
            Question.objects.create(
                quiz=quiz,
                question_text=request.POST.get('question_text', '').strip(),
                option1=request.POST.get('option1', '').strip(),
                option2=request.POST.get('option2', '').strip(),
                option3=request.POST.get('option3', '').strip(),
                option4=request.POST.get('option4', '').strip(),
                correct_answer=request.POST.get('correct_answer', '').strip()
            )
            return redirect('manage_questions')

    return render(request, 'add_question.html', {'quizzes': quizzes})


# Manage Questions: View all questions in the database
def manage_questions(request):
    questions = Question.objects.all().select_related('quiz').order_by('-id')
    return render(request, 'manage_questions.html', {'questions': questions})


# Edit Question: Update details of an existing question
def edit_question(request, question_id):
    question = get_object_or_404(Question, id=question_id)
    quizzes = Quiz.objects.all().order_by('name')

    if request.method == 'POST':
        quiz_id = request.POST.get('quiz')
        if quiz_id:
            question.quiz = get_object_or_404(Quiz, id=quiz_id)
        question.question_text = request.POST.get('question_text', '').strip()
        question.option1 = request.POST.get('option1', '').strip()
        question.option2 = request.POST.get('option2', '').strip()
        question.option3 = request.POST.get('option3', '').strip()
        question.option4 = request.POST.get('option4', '').strip()
        question.correct_answer = request.POST.get('correct_answer', '').strip()

        question.save()
        return redirect('manage_questions')

    return render(request, 'edit_question.html', {'question': question, 'quizzes': quizzes})


# Delete Question: Remove a question entirely
def delete_question(request, question_id):
    question = get_object_or_404(Question, id=question_id)
    question.delete()
    return redirect('manage_questions')


# View Results: Admin view to see scores of all students
def view_results(request):
    results = TestResult.objects.all().select_related('user', 'quiz').order_by('-attempted_at')
    return render(request, 'view_results.html', {'results': results})
```

4. Live Application Output Screenshots

Below are high-resolution screenshots documenting the complete QuizCraft AI platform: marketing landing, authentication, student exam catalog, assessment interface, and the administrator control center.



Figure 4.1: QuizCraft AI Public Landing Page with hero branding, features breakdown, and authentication gateways.

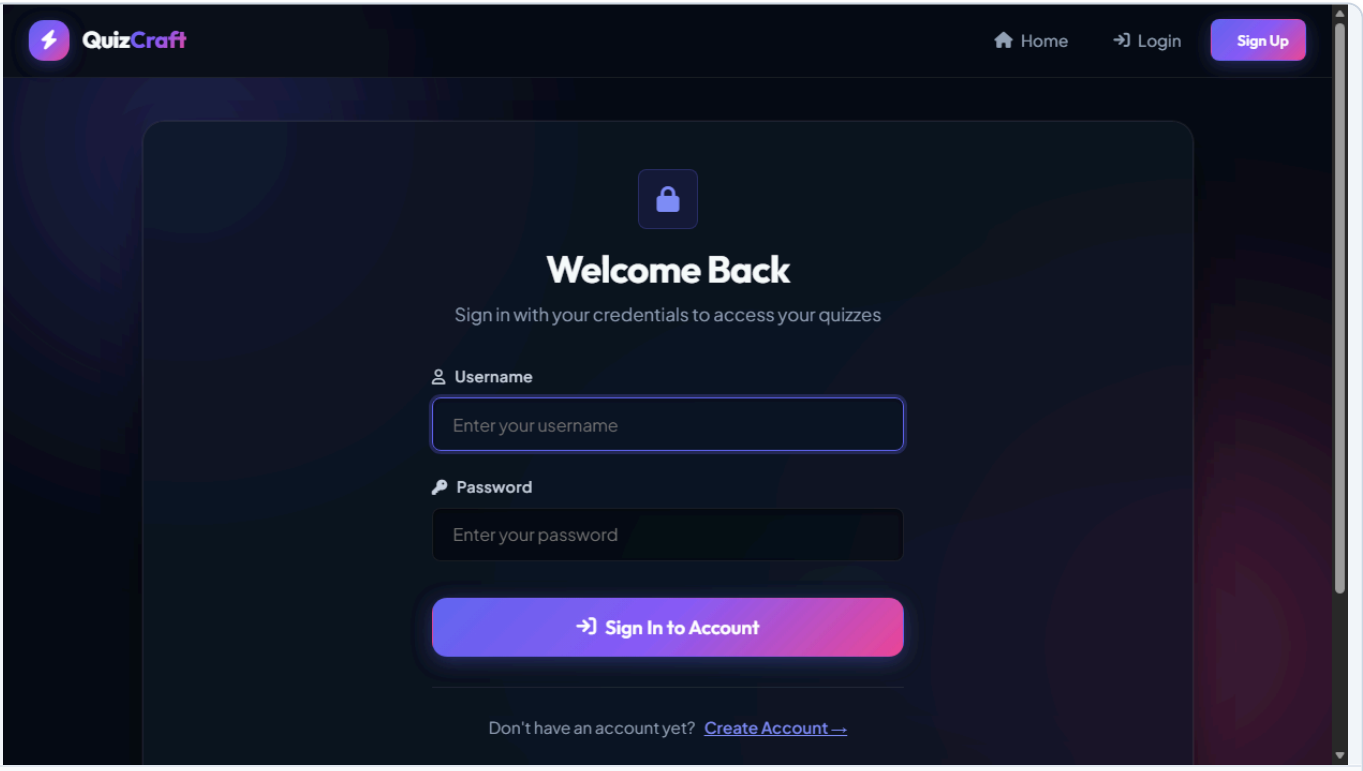


Figure 4.2: Clean Authentication Gateway for Students and System Administrators.

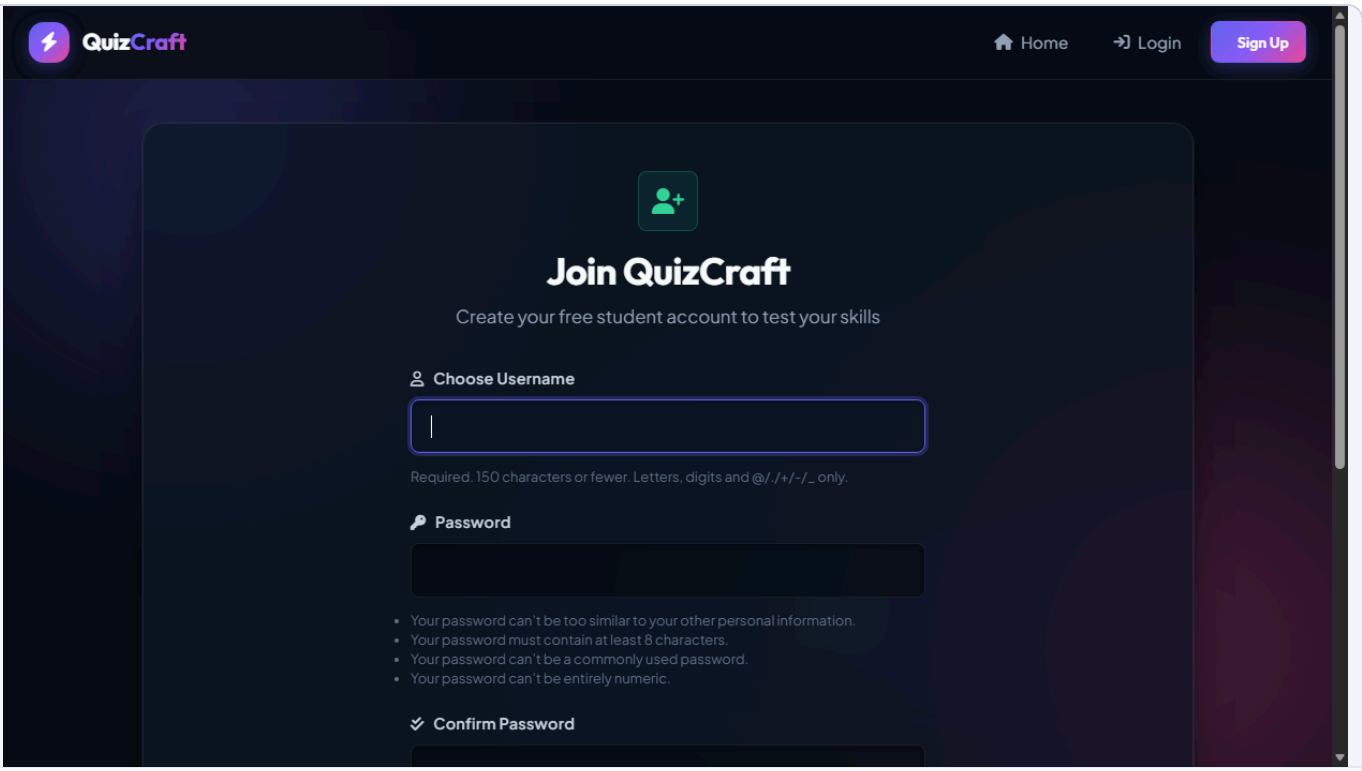


Figure 4.3: Student Account Registration Portal validating unique usernames and password confirmations.

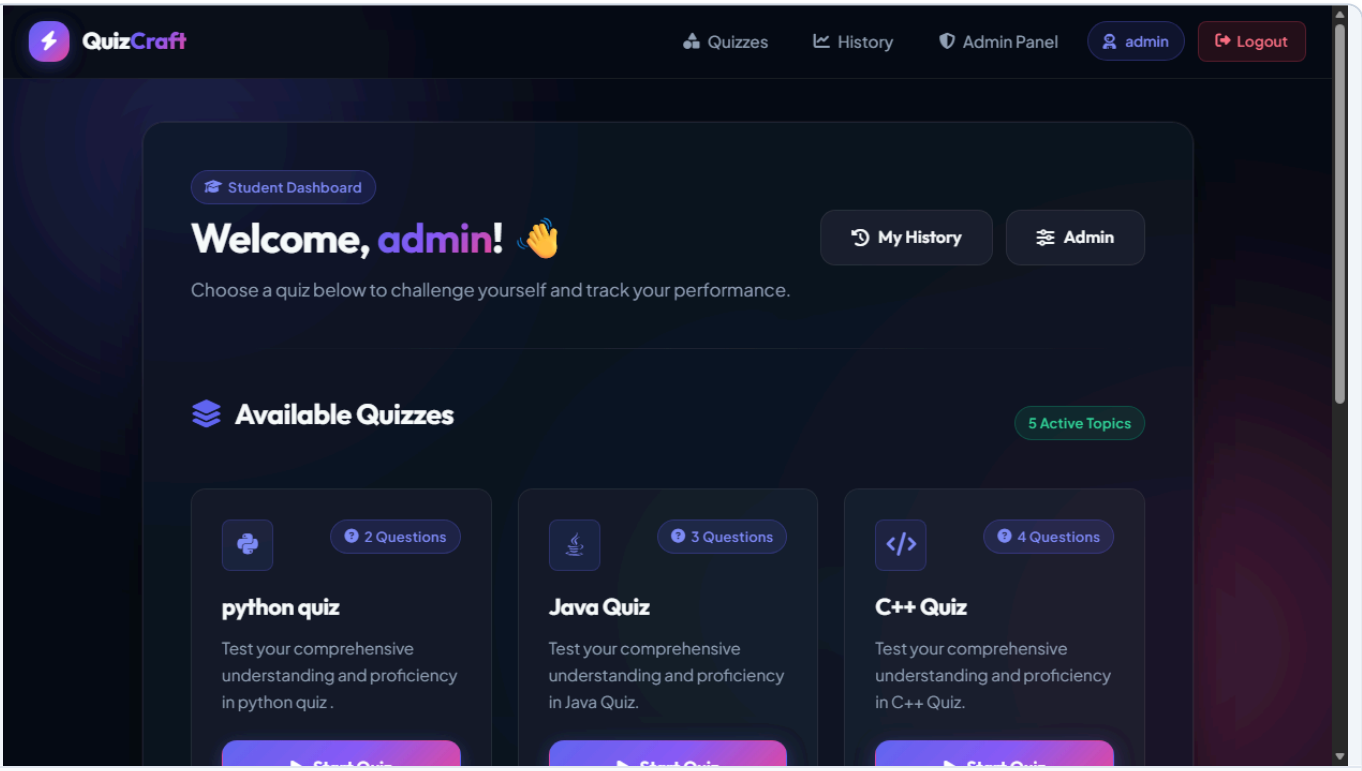


Figure 4.4: Student Examination Catalog displaying available quizzes (Python, Java, C++, Science) with question counts.

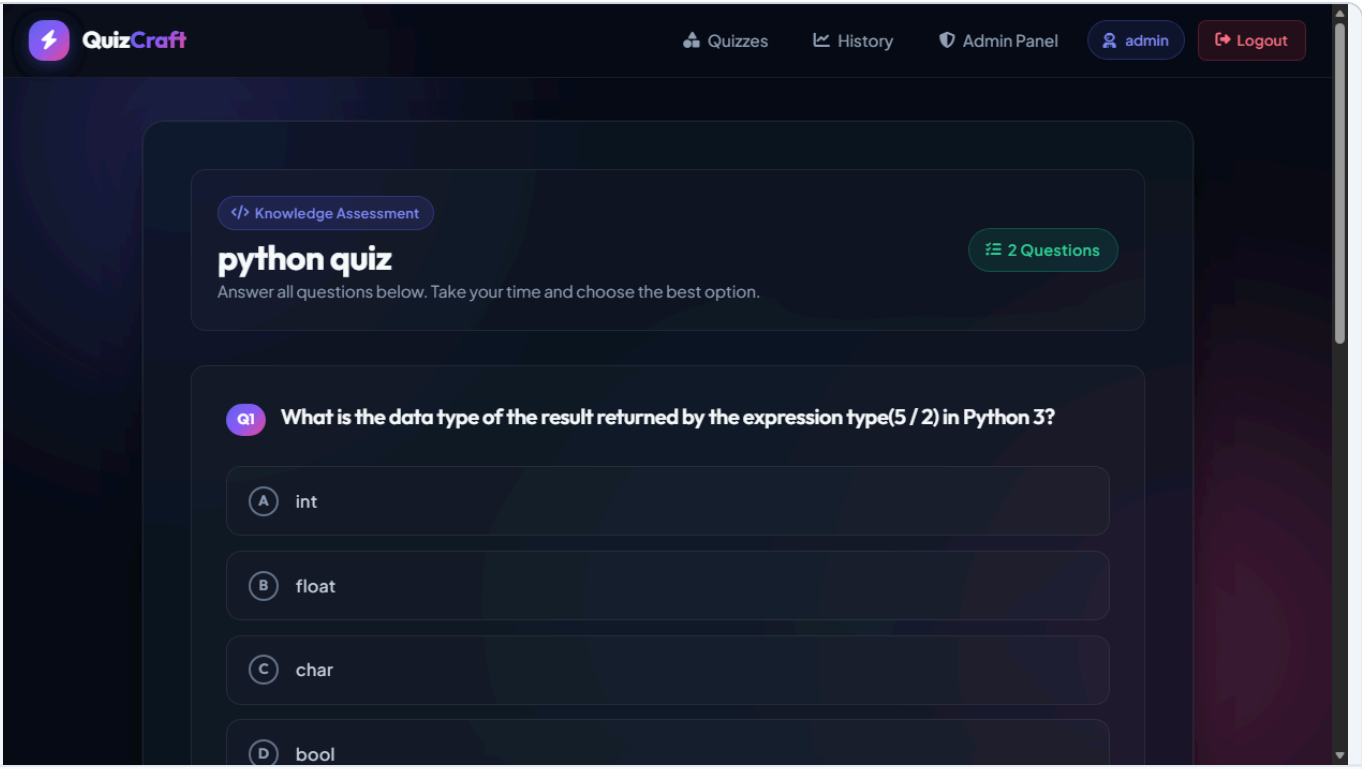


Figure 4.5: Interactive Assessment Interface displaying Multiple Choice Questions (MCQs) and dynamic selection options.

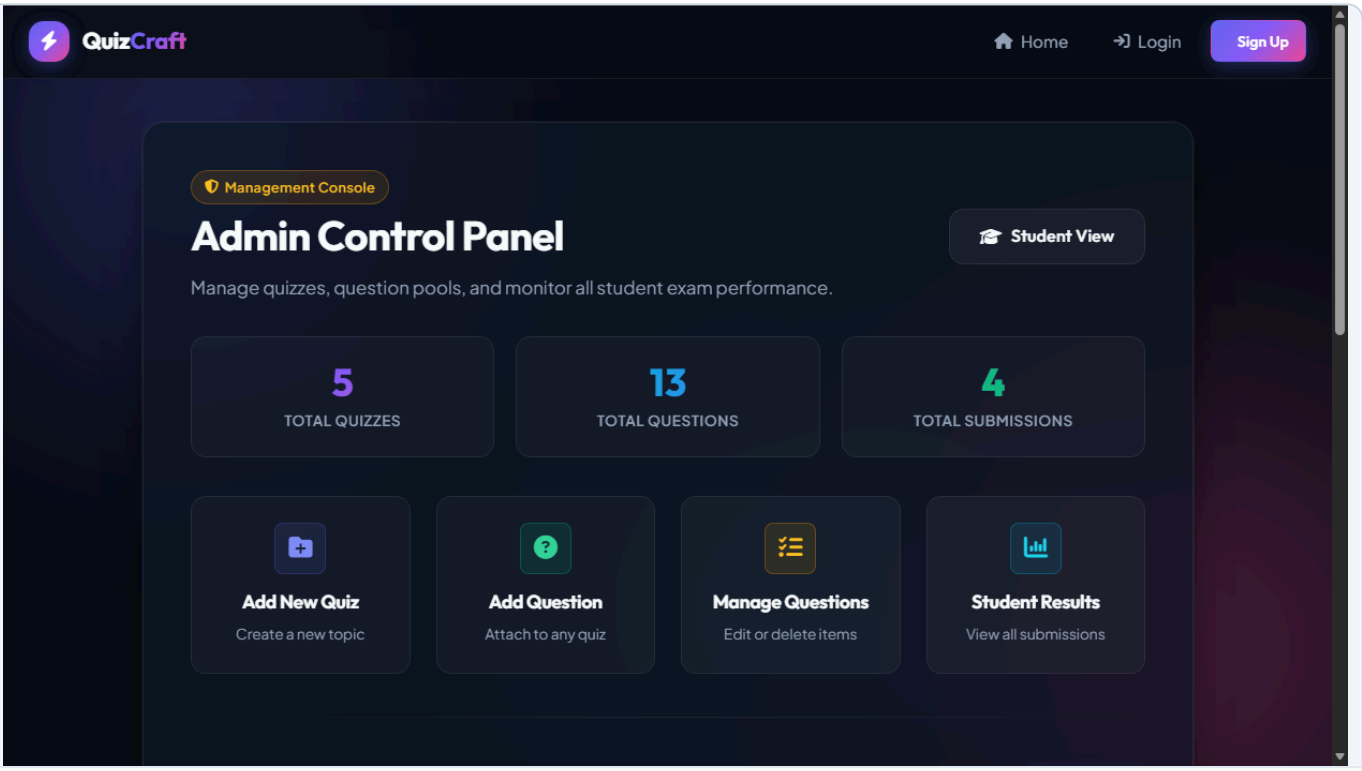


Figure 4.6: Administrator Control Panel displaying real-time KPI statistics (Total Quizzes, Questions, and Submissions) with management actions.

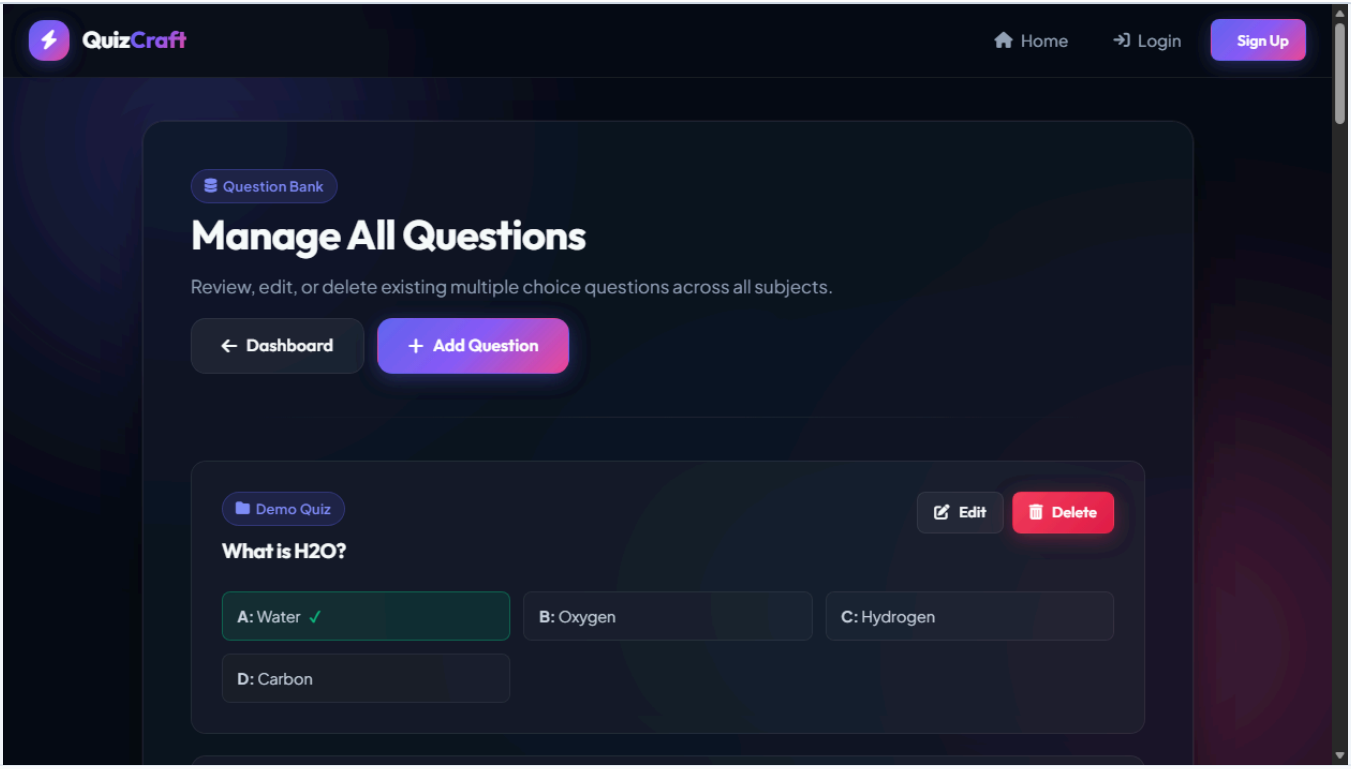


Figure 4.7: Question Pool Management Console allowing administrators to edit or delete question records across all quizzes.

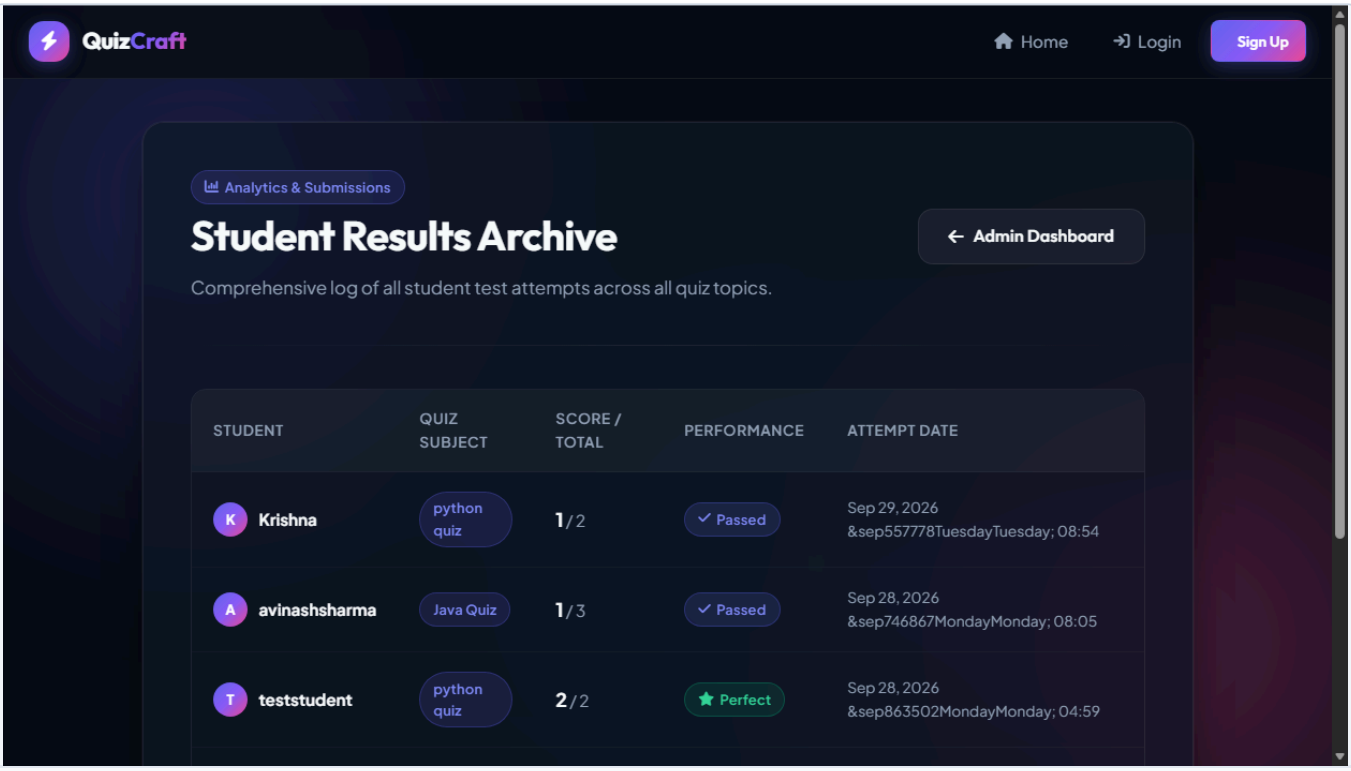


Figure 4.8: Student Results Archive showing comprehensive submission logs with student usernames, scores, total questions, performance badges, and timestamps.